

Lecture 2 • Week 1

The Thinking Machine: Fetch-Decode-Execute & Pipelining

From the moment a program starts running, the processor is executing a relentless loop — fetch an instruction, figure out what it says, act on it. This lecture makes that loop concrete and then asks: why do it one step at a time?

The Stored-Program Model

When you run `python script.py` and it starts executing, where is the program? How does the CPU know what to do first?

Von Neumann's 1945 insight was deceptively simple: store both the program instructions and the data they operate on in the same memory. Before this, computers were physically rewired to run different programs. The stored-program model means a program is just data — a sequence of numbers sitting in memory — and the CPU reads and executes those numbers in order.

The key components of the Von Neumann model are: a **CPU** (containing an ALU and a set of registers), a **unified memory** holding both instructions and data, and an **I/O subsystem**. The CPU and memory are connected by three buses: the **address bus** (CPU sends the address it wants to read/write), the **data bus** (the actual data flows here), and the **control bus** (read/write signals, clock, interrupts).

Inside the CPU, two special registers make the FDE cycle possible:

- **Program Counter (PC)**: holds the memory address of the next instruction to fetch. Automatically increments after each fetch.
- **Instruction Register (IR)**: holds the currently executing instruction while it's being decoded and executed.

The Fetch-Decode-Execute Cycle in Full Detail

Every instruction a CPU ever runs goes through this three-phase loop. Let's trace it precisely.

Phase 1: Fetch

The CPU reads the instruction at the memory address stored in the PC. Concretely: the PC's value is placed on the address bus, memory returns the instruction word on the data bus, and the instruction is loaded into the IR. The PC then increments by the instruction width (4 bytes on a 32-bit RISC machine).

Phase 2: Decode

The control unit examines the IR and figures out what to do. It reads the **opcode** (the field that names the operation — ADD, LOAD, BRANCH, etc.) and identifies which registers and/or memory addresses are involved. This phase produces a set of control signals that configure the datapath for the operation about to be executed.

Phase 3: Execute

This is the broadest phase. Depending on the instruction type, several things can happen:

- **Arithmetic/Logic:** the ALU performs the operation and writes the result to a register.
- **Load (Memory Read):** an address is computed, the CPU reads that memory location, and the result is written to a register.
- **Store (Memory Write):** an address is computed, and a register's value is written to that memory location.
- **Branch:** a condition is checked; if true, the PC is overwritten with a new address (the branch target) instead of just incrementing.

After execution, the cycle repeats from Fetch — now with the updated PC.

Worked example — tracing ADD R3, R1, R2

Assume $R1 = 5$, $R2 = 7$. The instruction `ADD R3, R1, R2` is stored at memory address `0x1000`, encoded as a 32-bit word.

Fetch: $PC = 0x1000$. CPU puts `0x1000` on address bus. Memory returns `0x00221820`. $IR \leftarrow$ 32 bit instruction. $PC \leftarrow 0x1004$.

Decode: Control unit reads top 6 bits (opcode). Reads rs field ($R1$), rt field ($R2$), rd field ($R3$), funct field (ADD). Control signals: ALU-op=ADD, write-enable on $R3$, read $R1$ and $R2$.

Execute: ALU receives 5 and 7. ALU outputs 12. Register file writes $12 \rightarrow R3$. Flags updated (zero flag = 0, carry = 0).

Loop restarts. PC is now `0x1004` — the next instruction.

What About More Complex Instructions?

A **Load** instruction like `LW R1, 8(R2)` (MIPS: "load word at address R2+8 into R1") adds an extra step: after the execute phase computes the effective address (R2+8), the CPU performs a second memory access to actually read the data. This is why a simple FDE model is often extended to 5 phases in practice — more on this when we discuss pipelining.

The Problem with Sequential Execution

In a naive **single-cycle processor**, every instruction takes exactly one clock cycle — but that clock cycle must be long enough for the slowest possible instruction. If a memory load takes 5ns and an ALU add takes 1ns, the clock period must be 5ns even for add instructions. The add instruction is sitting idle for 4ns of every 5ns cycle. This is a $5\times$ waste on the most common operation in any program.

A **multi-cycle processor** helps: different instructions take different numbers of cycles. An ALU op might take 3 cycles, a load 5 cycles. Now the clock can run faster (each cycle is shorter), and simple instructions finish sooner. But there's still a fundamental problem: only one instruction is *in flight* at any moment. All the other hardware sits idle while one instruction goes through its phases.

The motivating bottleneck

Consider a factory that builds cars one at a time: welding → painting → assembly → testing. While painting, the welding station sits empty. The throughput is 1 car per (welding + painting + assembly + testing) time. The solution used in every modern factory — and every modern CPU — is the assembly line: overlap the stages across multiple work items simultaneously.

Pipelining: The Assembly Line for Instructions

A **pipelined processor** divides instruction execution into stages and processes multiple instructions simultaneously — each in a different stage. The classic RISC pipeline has **5 stages**:

- **IF — Instruction Fetch:** Read instruction from memory (or instruction cache) using the PC.
- **ID — Instruction Decode / Register Read:** Decode the opcode, read source registers from the register file.
- **EX — Execute:** ALU performs the operation; for branches, compute the target address.
- **MEM — Memory Access:** Load instructions read from data memory; store instructions write to it. ALU instructions pass through.
- **WB — Write Back:** Write the result (from ALU or memory) back to the destination register.

Each stage is implemented by dedicated hardware. Pipeline **registers** (also called interstage latches) sit between each pair of stages — they hold the intermediate values produced by one stage so the next stage can process them on the following clock cycle. This means after an initial fill-up latency of 5 cycles, the pipeline produces **one instruction result per cycle** in steady state.

Pipeline Timing Diagram

The best way to internalize pipelining is through the timing diagram. The rows are instructions; the columns are clock cycles. Each cell shows which stage that instruction is in during that cycle.

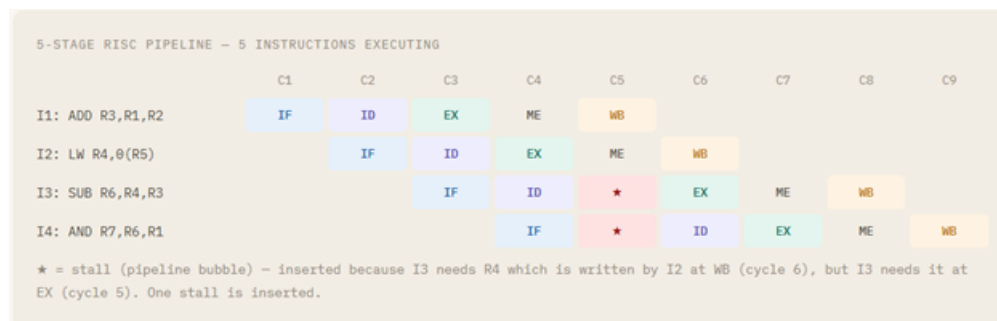


Figure 1: Example pipeline timing diagram

In steady state (ignoring the example stall above), the ideal CPI is 1. Compare that to single-cycle (CPI = 5 for a 5-stage pipeline). For a program with N instructions, the pipelined execution time is approximately $(N + 4)$ cycles vs. $5N$ cycles for single-cycle. As N grows large, the speedup approaches $5\times$.

Throughput Comparison

$$\text{Speedup} = \frac{N \times k}{k + N - 1} \approx k \quad \text{for large } N$$

k = number of pipeline stages. For large N (many instructions), $\text{speedup} \rightarrow k$.

Pipeline Hazards: When the Pipeline Stalls

The pipeline assumes that each instruction can enter the execute stage the moment its predecessor moves out of decode. This assumption fails in three ways — these are called **hazards**.

1. Structural Hazards

Two instructions need the same hardware at the same time. Classic example: a load instruction needs the memory in the MEM stage, while a fetch needs memory in the IF stage simultaneously — but there's only one memory. Solution: use **separate instruction and data caches** (Harvard architecture at the cache level). Most modern processors do this. Structural hazards are the easiest to eliminate by adding hardware.

2. Data Hazards

An instruction needs data that a previous instruction hasn't finished producing yet. There are three types:

- **RAW — Read After Write (true dependency):** I2 reads a register that I1 writes. I2 cannot proceed until I1 has committed the result. *This is the most common and most important hazard.*
- **WAW — Write After Write:** Two instructions both write the same register; they must complete in order. Rare in simple in-order pipelines.
- **WAR — Write After Read:** I2 writes a register that I1 still needs to read. Only a problem in out-of-order processors.

Example RAW hazard

ADD R1, R2, R3 — writes R1 at the WB stage (cycle 5 if it enters IF at cycle 1)

SUB R4, R1, R5 — reads R1 at the ID stage (cycle 3 if it enters IF at cycle 2)

R1 is written at cycle 5 but needed at cycle 3. The hardware must either **stall** (insert 2 bubble cycles) or use **data forwarding**.

The Elegant Fix: Data Forwarding (Bypassing) Instead of waiting for a result to travel all the way to the register file (WB stage) before the dependent instruction can use it, we add **forwarding paths** — direct wires that carry the result from the output of the EX stage (or MEM stage) straight to the input of the EX stage for the dependent instruction. This eliminates most stalls for ALU-to-ALU dependencies.

The one case forwarding cannot fully cure: the **load-use hazard**. If instruction I1 is a LOAD and instruction I2 immediately follows and uses the loaded value, the data isn't available from memory until the end of the MEM stage of I1 — which is the same cycle that I2 needs it at EX. One stall cycle is unavoidable. The compiler can sometimes reorder instructions to fill this "load delay slot."

3. Control Hazards (Branch Hazards)

A branch instruction changes the PC — but the processor has already started fetching the next sequential instruction(s) before it knows whether the branch is taken and where the target is. If the branch is taken, those fetched instructions must be **flushed** (thrown away), wasting cycles.

With a 5-stage pipeline, the branch outcome is known at the end of the EX stage (cycle 3 of the branch instruction's lifetime), meaning 2 instructions may have been wrongly fetched. This is a 2-cycle branch penalty.

Solutions: **branch delay slots** (the processor always executes the 1 or 2 instructions after a branch regardless — the compiler must fill these slots with useful work), and more powerfully, **branch prediction**. Modern processors predict branch outcomes (taken/not-taken) with over 99% accuracy using sophisticated history tables. A misprediction causes a flush of the in-flight instructions — the penalty, on a deep pipeline, can be 15–20 cycles. This is the "branch misprediction penalty" — it's why the Pentium 4's extremely deep pipeline was so risky.

Pipeline Performance Analysis

The key metric for pipeline performance is **CPI — Cycles Per Instruction**.

CPI Formula (with hazards)

$$CPI = \text{Ideal CPI} + \text{Stalls per instruction}$$

Ideal CPI = 1. Stalls come from data hazards (load-use) and control hazards (branch mispredictions).

CPU Time Formula

$$\text{CPU Time} = IC \times CPI \times T_{\text{clk}}$$

IC = instruction count, T_{clk} = clock period (1 / clock frequency).

Aha — the tradeoff of deeper pipelines

A deeper pipeline (more stages) allows a higher clock frequency (each stage does less work, so the critical path is shorter). But the branch misprediction penalty grows with depth. Intel's Pentium 4 had a 31-stage pipeline, enabling 3.8 GHz — but every branch misprediction wasted a lot of cycles, and branch-heavy code ran slower than on a shorter pipeline at lower frequency. This is why modern processors (Intel Core, Apple M-series) use moderate pipeline depths (14–20 stages).